Build Your Own Auto-Scaling Controller

Challenge Based Learning No. 1

August 25, 2026

1 Introduction

A fundamental property of cloud computing is the ability to acquire and release computing resources according to the needs of an application. This ability, however, does not by itself make a system elastic. Elasticity requires a mechanism capable of observing the system, recognizing changes in demand, deciding how much capacity is required, and carrying out the corresponding actions at the right time.

In managed cloud services, much of this process is presented to users as a configurable feature. A user selects a metric, defines a policy, and the cloud provider modifies the available capacity. Although this abstraction is useful operationally, it hides essential questions: What does it mean for capacity to be sufficient? What information can be used to determine this? When should the system act? How much capacity should be added or removed? How can late, unnecessary, or unstable decisions be prevented?

In this challenge, each student must answer these questions by designing, implementing, and evaluating its own *Auto-Scaling Controller*. The controller must close the loop between the observed state of an application and the computing capacity allocated to it.

Finally, this is an individual engineering challenge. Each student must independently design, implement, and experimentally evaluate an auto-scaling controller for a cloud-based application.

2 Background and Context

2.1 Elasticity

Elasticity can be understood as the ability of a system to acquire and release resources automatically so that the supplied capacity matches current demand as closely and promptly as possible [1]. Elasticity and scalability are therefore not synonyms. Scalability expresses a system's ability to handle a larger workload by adding resources. Elasticity also incorporates time, automation, and the ability to reduce capacity when that capacity is no longer required.

This challenge focuses on horizontal scaling: the system will modify the number of instances serving the application. A perfectly elastic system would make capacity follow demand without delay, overprovisioning, or service degradation. In a real system, however, measurement, decision-making, provisioning, and instance initialization all take time. The controller must recognize and manage these limitations.

2.2 The Controller as a Control Loop

An elasticity controller operates as a feedback control system. It must continuously:

1. observe the state of the application and its infrastructure;
2. analyze whether the current capacity is adequate;
3. decide whether to maintain, increase, or reduce that capacity;
4. execute the decision on the infrastructure;
5. observe the result of the action and begin a new cycle.

The expected overall behavior can therefore be expressed as

$$\text{observe} \rightarrow \text{analyze} \rightarrow \text{decide} \rightarrow \text{act} \rightarrow \text{observe again.} \tag{1}$$

Amazon CloudWatch will serve as a source of operational information. AWS instances and services produce managed metrics, and the application may publish additional metrics when necessary [2]. Each student must decide what information its controller requires, how to obtain that information, and how to transform it into decisions.

3 Problem Statement

An organization has deployed a web application on virtual machines in AWS. User requests are distributed among the available instances by a load balancer. Demand changes over time and cannot be known precisely in advance.

When capacity is insufficient, latency and error rates may increase, and requests may be lost. When unnecessary capacity remains allocated, the organization incurs avoidable costs and wastes resources. The number of instances should therefore not remain fixed.

The organization has decided not to use AWS-managed dynamic scaling policies. Instead, it requires a custom component capable of autonomously determining the required capacity and acting on the infrastructure.

Central Question

> How can a controller maintain the expected application performance under changing demand while avoiding unnecessary capacity?

4 The Challenge

Each student must design, implement, and evaluate an **Auto-Scaling Controller** for the provided application. The controller must observe operational metrics available in Amazon CloudWatch and autonomously decide whether to:

- maintain the current capacity;
- increase the number of instances;
- reduce the number of instances.

Its decisions must produce actual changes in the infrastructure. Student may use AWS mechanisms to provision, configure, register, and remove instances. However, **the logic that transforms observations into scaling decisions must be designed and implemented entirely by the student**.

No specific metric, algorithm, evaluation frequency, execution model, or scaling policy is prescribed. These choices are part of the problem and must be researched, justified, and evaluated.

5 Connection to the Assigned Reading

This challenge builds on the concepts and taxonomy presented by Al-Dhuraibi et al. in *Elasticity in Cloud Computing: State of the Art and Research Challenges* in [4].

The purpose of the challenge is not merely to automate the creation and termination of EC2 instances. Each student must design and implement an elasticity mechanism and demonstrate how that mechanism relates to the concepts discussed in the assigned paper.

Before implementing the controller, each student must classify the proposed solution according to the relevant dimensions of the elasticity taxonomy presented in the paper. The classification must identify and justify, at least:

- the type and direction of elasticity;
- the resources being scaled;
- the scope of the elasticity mechanism;
- the purpose of the mechanism;
- its mode of operation;
- the decision-making method;
- the controller architecture;
- the cloud-provider scope.

The student must explain which dimensions are imposed by the challenge and which correspond to design decisions made during the development of the solution.

6 Objectives

6.1 General Objective

Build a horizontal scaling controller that automatically adjusts the capacity of an application deployed in AWS in response to changing demand, and evaluate its ability to balance performance, stability, and resource usage.

6.2 Specific Objectives

1. Identify metrics that represent demand, application state, and available capacity.
2. Design a policy that transforms system observations into scaling decisions.
3. Integrate the controller with Amazon CloudWatch and AWS provisioning mechanisms.
4. Conduct a reproducible experiment involving workload variations.
5. Analyze the resulting behavior in terms of performance, stability, timeliness of decisions, and resource utilization.

7 Scope and Constraints

The solution must meet the following conditions:

1. The application will run on Amazon EC2 instances behind a load balancer.
2. The system must operate with a minimum of one and a maximum of five instances.
3. A substantial portion of the information used by the controller must come from Amazon CloudWatch.
4. AWS-managed dynamic scaling policies may not be used to decide when or by how much to scale.
5. The controller must operate without human intervention during the experiment.
6. The solution must respond both to increases and decreases in demand.
7. Every decision must be recorded and explainable from the observed system state.
8. The controller must account for failures or incomplete information without jeopardizing application availability.
9. Permissions assigned to the controller must follow the principle of least privilege.

The purpose of this challenge is not to reimplement a hypervisor, load balancer, or the internal mechanisms AWS uses to create a virtual machine. The design object is the **controller**: its observation model, decision policy, interaction with the actuator, and behavior as a closed control loop.

8 Engineering Decisions

As part of the development process, each student must investigate and justify at least the following decisions:

- What does it mean for the available capacity to be adequate?
- Which metrics represent demand and application state?
- How and how often will those metrics be obtained?
- What observation interval should be considered before making a decision?
- Under what conditions should capacity be increased, reduced, or maintained?
- How many instances should be added or removed in each decision?
- Where will the controller execute, and how will it retain state?
- How will the time required for a new instance to become truly available be taken into account?
- How will repeated, contradictory, or oscillating actions be avoided?
- How will missing, delayed, or anomalous metrics be handled?
- What will happen when an AWS operation fails?
- How will the controller determine that removing capacity is safe?

These questions are not an implementation sequence. They identify engineering issues that must be resolved as part of the challenge.

9 Observable Behavior

During each evaluation cycle, the controller must explicitly produce one of the following decisions:

```
MAINTAIN_CAPACITY
INCREASE_CAPACITY
REDUCE_CAPACITY
```

Listing 1: Minimum decisions produced by the controller

The record of each decision must make it possible to reconstruct at least:

- the time at which the decision was made;
- the metrics and observation interval considered;
- the existing capacity and its state;
- the decision and its justification;
- the action requested on the infrastructure;
- the result of that action.

10 Deliverables

10.1 Solution Design

A document describing the architecture, components, information and action flows, control policy, permissions, assumptions, and limitations of the solution.

10.2 Reproducible Implementation

A repository containing the controller source code, infrastructure definition or configuration, deployment and execution instructions, the load-generation tool or procedure, and the decision-logging mechanism. Credentials must not be stored in the repository.

10.3 Experimental Evidence

Complete results from the workload scenario, including time series, decision logs, capacity changes, observed performance, and resource utilization.

10.4 Critical Analysis

A report explaining the controller's behavior, the strengths and weaknesses of its policy, compliance with the service-level objective, the capacity used, the limitations encountered, and possible improvements.

10.5 Demonstration

A live demonstration in which workload variation, an autonomous decision, the resulting capacity change, and its subsequent effect on the application can be observed.

11 Assessment Criteria

The assessment will not be limited to verifying that the system creates or terminates instances. It will consider how convincingly the student demonstrates that its controller observes, decides, and acts coherently, stably, securely, and explainably.

12 Questions for the Final Presentation

At the end of the challenge, each student must be able to answer:

1. Why do the selected metrics adequately represent the system state, and what relevant aspects do they fail to capture?
2. How much time elapses between the appearance of an overload condition and the availability of additional capacity?
3. How does the controller distinguish a transient variation from a sustained change in demand?

4. What prevents the controller from repeatedly increasing and reducing capacity?
5. What happens when a measurement or an infrastructure action fails?
6. In what scenario did the controller make an incorrect or late decision?
7. What was the resource cost of maintaining the service-level objective?
8. If two controllers maintain the same service level, but one uses more instances or performs more capacity changes, which one is better, and how can this be demonstrated?

References

[1] N. R. Herbst, S. Kounev, and R. Reussner, "Elasticity in Cloud Computing: What It Is, and What It Is Not," in *Proceedings of the 10th International Conference on Autonomic Computing (ICAC)*, 2013, pp. 23–27.

[2] Amazon Web Services, "Metrics in Amazon CloudWatch," *Amazon CloudWatch User Guide*. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/working_with_metrics.html

[3] Amazon Web Services, "Dynamic scaling for Amazon EC2 Auto Scaling," *Amazon EC2 Auto Scaling User Guide*. [Online]. Available: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scale-based-on-demand.html

[4] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, "Elasticity in Cloud Computing: State of the Art and Research Challenges," *IEEE Transactions on Services Computing*, vol. 11, no. 2, pp. 430–447, Mar.–Apr. 2018, doi: 10.1109/TSC.2017.2711009.